ABBA4 Implicit: Canonical Theory

One symmetric projection around an unprojected triple jump

Explicit palindromic operations

All ABBA and BM4 presets inherit `PalindromicMethod`. For signed $d = ch$, the elementary operations are

$$A_d(u,v,t) = (u + df(t,v), v, t), \qquad B_d(u,v,t) = (u, v + df(t,u), t), \qquad T_d(u,v,t) = (u, v, t + d).$$

Only T advances time. A coefficient $c = 1/2$ means duration $h/2$; there is no additional half inside A or B. The optional exact C flow rotates the copy difference and requires an independent coupling object. Both BM4 presets omit C.

In chronological order, standard ABBA2 is $A_{h/2}, B_{h/2}, T_{h/2}, T_{h/2}, B_{h/2}, A_{h/2}$. The distinct symmetric variant $T_{h/2}, A_{h/2}, B_{h/2}, B_{h/2}, A_{h/2}, T_{h/2}$ evaluates every spatial shear at the midpoint time. Both exhibit second order on the tested nonautonomous rotation. Arbitrary palindromes do not inherit fourth or sixth order, and an arithmetic projection need not be reversible or symplectic.

`Composition.from_half` reflects a supplied half sequence. The generic `PalindromicImplicit` and `PalindromicMidpoint` methods select one exterior Hairer projection or a final arithmetic mean. The standard signed ABBA/BM4 recipes, grouped observations and passive momentum normalization remain unchanged. NumPy and JAX execute the same explicit operations and differentiate their retained shear inputs. See `docs/models/extended/simulation/palindromic-operations.md`.

1 Unprojected fourth-order base

Let $\mathcal{A}_{q,t}$ be one signed endpoint-time ABBA map on two copies, with no intermediate diagonal projection. Define

$$\gamma = \frac{1}{2 - \sqrt[3]{2}}, \qquad \delta = -\frac{\sqrt[3]{2}}{2 - \sqrt[3]{2}}.$$

The complete base map is

$$\mathcal{C}_{h,t} = \mathcal{A}_{\gamma h, t+(\gamma+\delta)h} \circ \mathcal{A}_{\delta h, t+\gamma h} \circ \mathcal{A}_{\gamma h, t}.$$

Because $2\gamma + \delta = 1$ and $2\gamma^3 + \delta^3 = 0$, the palindromic composition cancels the leading third-order defect of a symmetric second-order base. The middle duration is negative, and the two copies pass continuously through all three factors.

2 One outer symmetric projection

For

$$E = \begin{pmatrix} I \\ I \end{pmatrix}, \quad P = \tfrac{1}{2}\begin{pmatrix} I & I \end{pmatrix}, \quad G = \begin{pmatrix} I & -I \end{pmatrix}, \quad N = G^T,$$

the reduced formulation is

$$M(\mu) = \mathcal{C}_{h,t}(Ez + N\mu), \tag{1}$$

$$r(\mu) = GM(\mu) + 2\mu = 0, \tag{2}$$

$$z^+ = P(M(\mu) + N\mu). \tag{3}$$

If J_j is the Jacobian of signed factor j, then $J_{\mathcal{C}} = J_3 J_2 J_1$ in execution order and

$$D_\mu r = G J_{\mathcal{C}} N + 2I.$$

Every residual evaluation traverses all three ABBA factors.

3 Configurations

Three signed unprojected ABBA maps enclosed by one spatial Hairer projection. Energy tracking is optional and passive. All three accepted signed ABBA traces contribute to the passive momentum. There is one nonlinear solve per complete step. The common state conventions are specified below.

4 Removed per-map implementation

The current default uses one outer projection. The former `after_each_abba_map` implementation, with three distinct roots and intermediate diagonal re-embedding, has been removed. It defined a different numerical map, since projection and factor evolution do not commute. An explicit request for that placement now raises an error. The legacy single-projection factory is removed. Use `ABBA4Implicit` for the single outer-projection method.

5 Properties and limitations

The ideal method is symmetric and designed for order four. Geometric claims refer to the converged projection root; finite nonlinear tolerances perturb the map. Negative internal time increments require the potential and dynamics to be evaluable slightly outside the outer interval. Observer records preserve the three ordered factors so the accepted-map tangent can be reconstructed.

6 Implementation correspondence

The public `ABBA4Implicit` configuration is in `src/methods/extended/abba.py`. One shared `solve_projection` call surrounds its complete six-stage recipe. There is no intermediate projection between the three ABBA pairs.

The public names, effective numerical defaults and exports from `simulation` remain unchanged. Internal imports use the defining modules; old import adapters have been removed. The ordinary implicit path is `advance`, `solve_projection`, `compose`, then explicit $A/B/T/C$ operations. Arithmetic midpoint variants use one explicit average instead of a nonlinear projection. Adjoint stages evaluate at the start of their signed substep; direct stages evaluate at its end.

Newton and Broyden share the spatial equations and convergence services. Exact particle-block tangents, passive energy and optional observation use retained shear inputs; energy is evaluated only after accepting the spatial map, with the existing factor $1/2$ for physical κ. Time and momentum never enter the spatial root. The new internal family name does not enable the retired full time/momentum projection.

Current module responsibilities and diagrams are documented in `docs/models/extended/simulation/extended-simulation-architecture.md`.

7 Explicit residual formulations and ideal-root tangent

Let $w = C(Ez + N\mu)$ and let a, b be the corrected final copies. The simultaneous unknown is (a, b, μ) and its equations are

$$F(a, b, \mu; z) = \begin{pmatrix} a - w_u - \mu \\ b - w_v + \mu \\ a - b \end{pmatrix} = 0.$$

Eliminating a, b gives $GC(Ez + N\mu) + 2\mu = 0$. Thus both formulations define the same ideal map; their finite-precision solve paths and work may differ. Newton uses the exact block derivative, while Broyden updates an approximate residual Jacobian. Neither method projects the intermediate factors.

At a converged reduced root, set

$$J_C = J_3 J_2 J_1, \qquad K = G J_C N + 2I, \qquad L = G J_C E.$$

For nonsingular K, the implicit function theorem gives

$$D_z \mu = -K^{-1} L, \qquad D_z z^+ = P J_C (E - N K^{-1} L).$$

The inverse denotes a linear solve in the implementation. This tangent is one outer implicit differentiation. A product of three independently projected physical tangents would describe the removed implementation instead. For independent particles, physical Jacobian blocks can be evaluated per particle, preserving the component-major physical layout.

8 State and energy strategies

Three signed unprojected ABBA maps enclosed by one spatial Hairer projection. Energy tracking is optional and passive. All three accepted signed ABBA traces contribute to the passive momentum. There is one nonlinear solve per complete step. The common state conventions are specified below.

9 Method instances and diagnostic meaning

The public class inherits physical initialization, advance and event construction from the common ABBA numerical class. Execution owns output and energy observation, and calls `IntegrationMethod.new_run` to create a fresh instance of the same concrete class for each execution, then starts the common driver. The instance owns its concrete state formulation, spatial projection equation and coefficients; no separate context or callback-based method record is needed. Each advance returns accepted state, small statistics and the typed trace of its single outer projection.

The common fixed controller owns the uniform accepted grid and independent shortened output advances. The common coordinator owns progress, collection, optional observation and final output. The collector stores requested samples and accepted metric rows, without retaining stage records by default. Export extracts physical coordinates and auxiliary energy, never another numerical projection.

One ABBA4 step has one accepted nonlinear solve and three unprojected ABBA factors per residual evaluation. Substep metric arrays consequently have one column; physical observation records have three unprojected factor snapshots and one spatial multiplier. Observer and shadow work do not alter main-step counters.

10 Verification and historical notes

The current implementation is checked for fourth-order convergence, reversibility, multiplier scaling, exact-root tangents, solver/formulation agreement and observer isolation. Architectural regression compares against the retained outer-projection implementation, not against the removed map. Nonlinear tolerances and floating-point precision limit formal properties; trajectory accuracy requires refinement, independently of energy diagnostics.

The per-map simultaneous-formulation and Jacobian companions are retained as historical derivations for interpreting earlier data. The active architecture is the shared extended-space diagram in the model's `simulation/` directory.

11 Physical accepted states and passive energy tracking

For N particles with d physical components, every accepted snapshot contains $Z \in \mathbb{R}^{dN}$ and one scalar time t. An optional energy observer owns separate physical momentum $\kappa \in \mathbb{R}^N$. Thus GC accepted vectors have $2N$ components and FC vectors have $4N$, whether energy is tracked or not. A doubled method prepares temporary $(u, v) \in \mathbb{R}^{2dN}$ copies, allows them to separate, and projects to one physical result before acceptance. The representation stores only the physical size and never owns a clock, momentum, physical coordinate declaration, dynamics or solver.

The normalized passive balance is

$$\dot{\kappa} = -\partial_t H(t, Z), \qquad \kappa(t_0) = 0, \qquad \Delta K(t) = H(t, Z(t)) + \kappa(t) - H(t_0, Z_0).$$

GC split-map contributions are summed in temporary work and divided by two to obtain physical momentum. Tracking leaves physical stages, nonlinear stopping scales and adaptive acceptance unchanged. An autonomous Hamiltonian supplies an explicit zero momentum derivative.

Hairer's constraint is solely $u - v = 0$: the reduced multiplier has $2N$ entries and ABBA's simultaneous spatial solve has $6N$ unknowns. Neither t nor κ is projected. The former eight-component projection is retired. Dynamics declare component order and position/velocity roles through immutable structural data; initial conditions and results retain those data independently. Execution injects $f(t, Z)$ and optional physical derivatives into a fresh method run. Methods own physical stages and projection equations, without retaining the dynamics object. Nonlinear tolerances, iteration limits, differentiation and iteration callbacks belong to the configured algebraic solver. Generic doubled copy operations live in `src/formulations/state.py`; classical, HBVM and adaptive methods need no state formulation.

Methods optionally return retained stage quadrature or a dense interpolant. The energy observer integrates momentum independently, using Hamiltonian functions supplied by execution. Controllers keep only physical accepted states; compiled JAX loops carry the observer accumulator separately. Observers receive physical states. Energy histories in `States.diagnostics` retain their existing labels; `extended_time` has shape (N, n_{samples}) and is broadcast from scalar sample times. Fixed shadow samples leave accepted states and counters unchanged. Adaptive quadrature uses accepted dense output and physical-only error control.

12 Optional JAX execution

An optional JAX driver executes the same numerical map on one selected CPU or GPU, with particle operations batched inside a compiled sequential time loop. It reuses the canonical coefficients and prepared potential splines. Physical-state tolerance scaling, signed stage times, passive energy weights and independent shortened output maps retain their definitions above. Applicable Newton systems use independent particle blocks; an explicitly selected Broyden iteration retains its global secant matrix. Nonlinear iteration limits and failure checks remain active on the device.

Both execution routes use double precision and return the same physical solution layout. Floating-point reassociation can produce round-off differences without changing the designed order or geometric properties. First-call compilation is separate from warmed integration timings, which include transfers and output construction. The sibling runtime document and shared execution contract specify supported callbacks and exact device scope.